

Unit 5 Notes: Project and DevOps (Docker)

Unit 5 Notes: Project and DevOps (Docker)

Contents

1. Course Orientation
 2. Why Docker Exists
 3. Containers vs Virtual Machines
 4. Docker Architecture
 5. Images and the Dockerfile
 6. Containers, Storage and Networking
 7. Docker Compose
 8. Integrating Django with Docker
 9. Deployment and Security
 10. Best Practice and Debugging
 11. Quick Recap
-

1. Course Orientation

Units 1 to 4 built an application. Unit 5 is about shipping it: making the same code run on a teammate's laptop, a college server and a cloud machine, exactly the same way, every time. Docker is the tool the industry uses to do that.

The unit follows one path: why the problem exists, how an application is packaged into an **image**, how that image runs as a **container**, how a whole stack is described with **Compose**, and finally how the Unit 2 Django project is containerised and deployed.

Code reference: every command and file mentioned here is in the Unit 5 Coding Reference. These notes cover the ideas behind it.

2. Why Docker Exists

2.1 "But it works on my machine"

An application is never just its code. It is code plus a runtime (a Python version), libraries (Django, psycopg2), operating system packages and configuration. Git ships only the code, and everything else drifts from machine to machine. The same commit can work on your laptop, fail with an ImportError on a teammate's Mac, and crash on a server with an older Python.

The problem appears even on a single laptop. A final year project might need Django 3.2, Python 3.8 and PostgreSQL 12, while an internship project needs Django 5.2, Python 3.12 and PostgreSQL 16. Both cannot be installed side by side in the normal way, a situation often called dependency hell.

The usual fixes each fall short:

Fix	What it isolates	Where it falls short
venv	Python packages only	Not PostgreSQL, not system libraries, not the OS
README setup steps	Nothing	Goes out of date, and one step always fails on Windows
Virtual machine	Everything	Gigabytes of disk, minutes to boot
Container	Everything the app needs	Megabytes, starts in about a second

2.2 What Docker is

Docker packages an application with everything it needs to run (code, runtime, libraries, OS files, config) into a standard, portable unit called an **image**, and runs that image as an isolated process called a **container**.

The idea is borrowed from the shipping container. Before 1956, cargo was loaded onto ships by hand, sack by sack and barrel by barrel, and a ship could take a week to load. Then one standard steel box arrived, and every crane, ship, train and truck handled it the same way without caring what was inside. Loading costs fell from about \$5.86 to \$0.16 per ton.

Shipping	Software
The cargo	Your app, plus Python, libraries and config
The standard steel box	A container image
Ship, train, truck	Laptop, college server, cloud machine
The port crane	The Docker Engine

Teams adopt Docker for a handful of reasons:

- **Consistency:** the same image runs on a laptop, in a CI pipeline and in production, which turns "works on my machine" into "works on every machine".
- **Speed:** there is no operating system to boot, so containers start in about a second.
- **Density:** containers share one kernel, so a laptop runs dozens where it could manage only two or three virtual machines.
- **Isolation:** each app has its own Python, libraries and database version, with no conflicts.

- **Ecosystem:** Docker Hub has ready made official images for PostgreSQL, Redis, nginx, Python and thousands more.

Docker is **not** a virtual machine, not a programming language, and not a replacement for Git or for the venv you use inside your editor.

3. Containers vs Virtual Machines

A **virtual machine** virtualises the hardware: each VM boots its own full operating system with its own kernel. A **container** shares the host's kernel and ships only its own files.

	Virtual machine	Container
Kernel	Its own, one per VM	Shared with the host
Size	Gigabytes	Megabytes
Start time	Tens of seconds to minutes	About a second
Per laptop	2 or 3	Dozens
Isolation	Strong, hardware level	Process level

A container is just a process. It is an ordinary Linux process that the kernel restricts using two built in features:

- **Namespaces** control what the process can **see**: its own process list (where it is process 1), its own network, hostname and filesystem.
- **Control groups (cgroups)** control what it can **use**: limits on CPU, memory and number of processes.

No operating system boots, so a container starts as fast as any program, and when its main process exits, the container stops.

Containers and virtual machines are not either/or. In the cloud, containers usually run inside virtual machines: the VM keeps different customers apart, and containers pack each customer's apps efficiently.

Note: Linux containers need a Linux kernel. On Windows and macOS, Docker Desktop quietly runs a small Linux virtual machine (WSL2 on Windows) and runs containers inside it. That is why a Python script that reports "Windows 11" on your laptop reports "Linux" inside a container.

4. Docker Architecture

Docker is split across three places:

- **Client:** the `docker` command you type. It only sends requests.

- **Daemon:** a background service that does the actual work: building images, running containers, managing storage and networks.
- **Registry:** a server that stores and shares images, such as Docker Hub. Images are pulled from it and pushed to it.

Five objects come up every day:

Object	What it is
Image	A read only template, built from a Dockerfile
Container	A running (or stopped) instance of an image
Volume	Storage managed by Docker that outlives containers
Network	A virtual switch; containers on it reach each other by name
Registry	Where images are stored and shared

An image and a container relate the way a class and an object do in Unit 2: one image, defined once, can produce any number of independent containers.

4.1 What happens when you type docker run

A single `docker run` command sets off a fixed sequence:

1. **Request:** the client asks the daemon to create and start a container.
2. **Image lookup:** the daemon checks whether the image is already in its local cache.
3. **Pull:** if not, it downloads each missing layer from the registry.
4. **Create:** it stacks the layers and adds a thin writable layer on top.
5. **Network:** it gives the container its own network interface and sets up any published ports.
6. **Start:** the kernel sets up namespaces and cgroups, and the app starts as process 1.

5. Images and the Dockerfile

5.1 Images are layers

An image is a stack of read only **layers**, one for each Dockerfile instruction that changes files, plus a small config recording the default command, environment variables and ports. Layers are identified by their content, so identical layers are shared between images and cached rather than downloaded twice. Images are **immutable**: changing one actually produces a new image.

Note: only instructions that change files, `RUN`, `COPY` and `ADD`, add size to an image. Instructions like `CMD` and `EXPOSE` only record settings. When an image is too big, the `RUN` and `COPY` layers are where to look.

5.2 Names, tags and "latest"

A full image name has several parts, most with defaults:

Part	Example	Default
Registry	ghcr.io	docker.io (Docker Hub)
Namespace	/yourname	library, for official images
Repository	/imap-django	(required)
Tag	:1.0	latest

So typing `postgres:16-alpine` really means `docker.io/library/postgres:16-alpine`. The image can also be referred to by its **digest**, a hash of its exact content, which can never change. A **tag**, by contrast, is only a label, and the publisher can move it to new content at any time. `latest` does not mean stable; it just means whatever was pushed most recently. A Dockerfile built on `python:latest` can work today and fail next month when `latest` moves to a new Python.

Rule: always pin at least the major and minor version, such as `python:3.12-slim`.

5.3 Choosing a base image

The base image decides most of an image's size. Measured in the lab, the same app was **1.65 GB** on `python:3.12` and **235 MB** on `python:3.12-slim`: seven times smaller, with identical code.

Flavour	Use when
Full (python:3.12)	You need compilers, and only in a build stage
slim	The default choice for Python apps
alpine	Tiny, but some Python packages do not install on it cleanly
distroless	Production images with no shell or package manager, so very little to attack

5.4 The Dockerfile

A Dockerfile is the recipe for an image, one instruction per line, run top to bottom. A typical one starts `FROM` a base image, sets a `WORKDIR`, `COPY`s files in, `RUN`s commands such as installing packages, and ends with the `CMD` the container runs.

Order matters because of the cache. Docker reuses a cached layer as long as that instruction and everything before it are unchanged; once one layer changes, every later layer is rebuilt. So `requirements.txt` is copied and installed **first**, and the source code is copied **last**. Code changes on nearly every build, dependencies rarely do, so the slow install stays cached. In the lab, a code edit rebuilt in 1.0 second with the right order and 5.4 seconds with the wrong one.

Three pairs are easy to confuse:

- **CMD vs ENTRYPOINT:** ENTRYPOINT is the program; CMD is its default arguments. Anything typed after the image name in `docker run` replaces CMD.
- **COPY vs ADD:** COPY just copies. ADD also unpacks archives and downloads URLs. Prefer COPY.
- **ARG vs ENV:** ARG exists only while building; ENV is baked into the image and seen by the app. **Neither is safe for secrets**, since both can be read from the image.

The build context and .dockerignore. The `.` at the end of `docker build` is the folder sent to the builder. Without a `.dockerignore` file, `COPY . .` would copy the venv, the `.git` history and the `.env` file with its passwords into the image.

Multi stage builds use one stage with all the build tools and a second, slim stage that copies in only the finished result, so build tools never reach production. For compiled languages the saving is dramatic: a React app built with Node (about 1 GB) can ship as static files on nginx (about 50 MB). For Python the size barely changes, but the final image is cleaner and safer, with no compilers left for an attacker to use.

6. Containers, Storage and Networking

6.1 Containers are disposable

Any file a container changes lives in its own thin writable layer, which is deleted along with the container. In the lab, votes stored inside a container vanished the moment the container was removed and recreated. This is correct behaviour, and it gives the central rule of Docker: **code belongs in images, data belongs in volumes.**

A container moves through a simple lifecycle: **created, running, exited**, and finally **removed**. `docker run` does the first two in one step. An exited container still exists, with its logs and writable layer, until it is removed: `docker ps` shows running containers, while `docker ps -a` shows all of them. The `--rm` option removes a container automatically when it exits, which suits one off commands.

6.2 Reading a docker run command

The command used in Unit 2 to start PostgreSQL shows the most common options:

Option	Meaning
<code>--name imap-postgres</code>	A readable name instead of a random ID
<code>-e POSTGRES_PASSWORD=...</code>	Sets an environment variable inside the container
<code>-p 5432:5432</code>	Publishes a port, host on the left, container on the right
<code>-d</code>	Runs in the background
<code>postgres:16</code>	The image and tag to run

Rule: options go **before** the image name. Anything written **after** the image name is the command to run inside the container, replacing the image's default.

6.3 Storage

Data that must outlive a container is mounted into it from outside:

	Volume	Bind mount	tmpfs
Managed by	Docker	You, it is a host folder	Memory
Best for	Databases, uploads	Source code during development	Temporary scratch files
Survives removing the container?	Yes	Yes	No

With a named volume mounted where the app writes its database, the votes survived removing and recreating the container.

Note: a bind mount gives a container real access to a folder on your laptop. In the lab, a container deleted its own entire filesystem without touching the laptop, but only because nothing was mounted. Isolation is only as good as what you share into the container.

6.4 Ports

A container has its own network, so an app inside it is not reachable from the laptop until a port is **published** with `-p HOST:CONTAINER`, read as "outside : inside".

- `EXPOSE` in a Dockerfile only documents a port; `-p` actually publishes it.
- An app inside a container must listen on `0.0.0.0`, not `127.0.0.1`. Inside a container, `127.0.0.1` means "this container only", so outside traffic is refused.
- "Port is already allocated" means another program on the laptop already uses that host port; pick another, such as `-p 8001:8000`.

6.5 Containers find each other by name

Containers on the same **user defined network** reach each other by name, because Docker runs a small DNS server on that network. A web container connects to its database simply as `db:5432`. Names stay the same across restarts; IP addresses do not.

Note, the localhost trap: inside the web container, `localhost` means the web container itself, and there is no database there. This is the most common beginner bug.

You are here	To reach PostgreSQL, use
Your laptop, with the port published	localhost:5432

You are here	To reach PostgreSQL, use
Another container on the same network	db:5432, the service name
A container, reaching something on your laptop	host.docker.internal:5432

This is exactly why Unit 2's `HOST: "localhost"` must change once Django runs in a container.

6.6 One image, many configurations

In the lab, the same image stored votes in SQLite when started with no settings, and in PostgreSQL when started with a `DATABASE_URL` environment variable. Nothing was rebuilt. Configuration comes from the environment, not the code, so one image serves development, testing and production.

7. Docker Compose

Starting a web app and its database by hand takes several long commands: create a network, create a volume, then start each container with the right names, variables and ports, in the right order. **Docker Compose** replaces all of that with one file, `compose.yaml`, and one command, `docker compose up`.

The file is **declarative** (it describes the end state, not the steps), **version controlled** alongside the code, and **reproducible**: a teammate starts the whole stack with one command. Compose creates the network and volumes itself, and each **service name becomes a hostname**.

A compose file lists **services**, each of which becomes a container, and the **volumes** they use. The keys map directly onto `docker run` options:

Key	Equivalent to
build / image	Building an image, or naming one to pull
ports	<code>-p</code>
environment	<code>-e</code>
volumes	<code>-v</code>
restart	<code>--restart</code>
depends_on	Start order between services

Three ideas matter most:

- **Secrets in .env:** Compose reads `.env` from the same folder and fills in `${VARIABLE}` values. `.env` is kept out of Git and out of images; a `.env.example` with placeholder values is committed instead.
- **Started is not ready:** a database container can be running while PostgreSQL inside it is still starting up. A **healthcheck** tests real readiness, and `depends_on` with `condition: service_healthy` makes the web service wait for it.

- **down vs down -v:** `docker compose down` removes containers and the network but keeps volumes, so data survives. `down -v` **also deletes the volumes**, wiping the database.
-

8. Integrating Django with Docker

Every idea above applies directly to the Unit 2 Django project. Four things change when Django moves into a container:

- **gunicorn replaces runserver.** `runserver` is a development tool. In a container, Django is served by **gunicorn**, a production server, pointed at the project's `wsgi.py` and bound to `0.0.0.0:8000`.
- **Settings come from the environment.** `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS` and the database password move out of `settings.py` into `.env`, so the same image works everywhere and no secret is baked into it. The database `HOST` becomes `db`, the Compose service name.
- **Migrations run at start up, not build time.** The database does not exist while the image is being built, so `migrate` runs when the web container starts, before gunicorn. The superuser is created inside the running container.
- **Static files need serving.** With `DEBUG` off, Django stops serving CSS itself, which is why the admin site suddenly looks unstyled. The files are gathered with `collectstatic` and served by a small package, WhiteNoise.

The trained `.pkl` model from Unit 2 travels inside the image with the code, and the scikit-learn version in the image must match the one that trained it, or the model may fail to load.

The finished stack has two services: **web**, built from the Django Dockerfile and published on port 8000, and **db**, the official PostgreSQL image with a named volume and a healthcheck. One `docker compose up` gives any teammate the full Unit 2 application, API and ML endpoint included, with no local Python or PostgreSQL at all.

9. Deployment and Security

9.1 The image is the deliverable

Deploying with Docker is three steps: **tag** the image with a registry name, **push** it, then **pull and run** it anywhere. In the lab, a classmate ran a pushed image on their own laptop with one command, with no code, Python or pip installed. The code, runtime and libraries travel together inside the image.

Common registries are **Docker Hub** (the default), **GitHub Container Registry** (next to your code), and cloud registries such as Amazon ECR.

A sensible tagging strategy:

Tag	Example	Changes?	Use it for
Version	1.0, 1.4.2	Never	Releases, what production runs

Tag	Example	Changes?	Use it for
Git commit	sha-3f2a9c1	Never	Tracing an image back to its exact code
Branch	main, dev	Every push	Test and staging environments
latest	latest	Every push	Convenience only

Rule: never push different content under a version tag that already exists, and never run `latest` in production. If `1.0` can change, nobody can say what is actually running.

Note: Apple M series Macs build arm64 images, while most servers are amd64. Running the wrong one gives "exec format error"; `docker buildx` can build for both at once.

9.2 CI/CD

Continuous Integration and Delivery automates the path from code to server. On every `git push`, a pipeline such as GitHub Actions starts a fresh machine and runs the same steps every time:

1. **Build** the image, reusing cached layers.
2. **Test** it by starting it and checking that it responds.
3. **Scan** it for known security vulnerabilities.
4. **Push** it to a registry with proper tags.
5. **Deploy** it by having the server pull the new image and restart.

Nothing depends on anyone's laptop, and every release is built the same way.

9.3 A production stack

A real deployment adds a few pieces around the app:

- **A reverse proxy:** nginx is the only service open to the outside, and forwards requests to the app.
- **Scaling:** the app runs as several identical copies, and nginx spreads requests across them.
- **Separate networks:** the proxy sits on a front network and the database on a back one, so the proxy cannot even reach the database.
- **Limits:** each copy gets a fixed share of CPU and memory, so one cannot starve the others.

9.4 Security

Every container on a host shares one kernel, so containers are strongly isolated but not as strongly as virtual machines. The key habits:

- **Never put secrets in an image.** A password set with `ENV` in a Dockerfile can be read by anyone with `docker inspect`, and deleting it in a later layer does not remove it from the earlier one. Secrets are supplied at run time, through `.env`.
- **Run as a non root user**, created in the Dockerfile.

- **Use a minimal, pinned base image**, since fewer packages means fewer vulnerabilities.
 - **Use `.dockerignore`** to keep `.env`, `.git` and keys out of the build.
 - **Limit resources** so one container cannot starve the rest.
 - **Scan images and rebuild regularly** to pick up security fixes in the base image.
 - **Never run containers with `--privileged`**, and never mount Docker's own control socket into a container; either effectively hands over the whole machine.
-

10. Best Practice and Debugging

10.1 Rules worth memorising

1. Pin a slim base image, such as `python:3.12-slim`.
2. Copy dependencies first and code last.
3. One concern per container: web, database and cache are separate services.
4. Bind to `0.0.0.0` inside containers.
5. Use `.dockerignore` for everything the image does not need.
6. Pass configuration through environment variables.
7. Log to the screen, not to files, so `docker logs` can show it.
8. Keep data in volumes, never in the container.
9. Run as a non root user.
10. Add a healthcheck, and make dependent services wait for it.

10.2 The debugging playbook

When a container does not work, check in this order:

1. **Is it running?** `docker ps -a` shows the status and exit code.
2. **What does it say?** `docker logs` shows the real error. Most problems end here.
3. **How is it configured?** `docker inspect` shows ports, environment variables, mounts and networks.
4. **Look inside.** `docker exec` opens a shell in the running container.
5. **Check the network.** Same network? Service name, not localhost? App bound to 0.0.0.0?

Error	Usual cause
port is already allocated	Another program already uses that host port
connection refused to the database	Using localhost instead of the service name, or the database is not ready yet
exec format error	Image built for a different CPU architecture
Exited (137)	Out of memory

Error	Usual cause
no space left on device	Old images and build cache; run <code>docker system prune</code>
pull access denied	A typo in the image name, or a private repository
Database is uninitialized and superuser password is not specified	PostgreSQL started without <code>POSTGRES_PASSWORD</code>

11. Quick Recap

- **Why Docker:** Git ships only code; an image ships code, runtime, libraries and config together.
- **Containers vs VMs:** containers share the host kernel, so they are megabytes and seconds, not gigabytes and minutes.
- **A container is a process:** namespaces limit what it sees, cgroups limit what it uses.
- **Architecture:** the client asks, the daemon works, the registry stores images.
- **Images:** immutable, cached layers; pin a slim base, because latest is not a version.
- **Dockerfile:** dependencies first, code last; `.dockerignore` keeps secrets and junk out.
- **Containers are disposable:** code in images, data in volumes.
- **Networking:** `-p` publishes, bind to 0.0.0.0, and containers use service names, never localhost.
- **Compose:** the whole stack in one file; healthchecks mean ready; `down -v` deletes data.
- **Django in Docker:** gunicorn, settings from `.env`, database host `db`, migrations at start up.
- **Deployment:** tag, push, pull, run; version tags never change; CI/CD automates build, test and push.
- **Security:** no secrets in images, non root user, minimal pinned base, scan and rebuild regularly.